

Taller 2: El intérprete de Cauca
"Un procedimiento es código más el ambiente donde nació"
Fundamentos de Interpretación y Compilación de Lenguajes de Programación

Carlos Andres Delgado S, MSc
carlos.andres.delgado@correounivalle.edu.co

Octubre de 2026

1. Reglas

- El taller se publica el **martes 6 de octubre de 2026**. La fecha límite de entrega es el **lunes 26 de octubre de 2026 a las 17:58:59**. Se califica el último commit anterior a esa fecha y hora; lo que se suba después no se tiene en cuenta.
- Los grupos deben tener máximo 4 personas.
- Debe usarse `#lang eopl` y la librería SLLGEN para la especificación léxica y gramatical.
- Las funciones deben definirse con **lambda**.
- **El grupo trabaja sobre un fork del repositorio base** `https://github.com/EjerciosClasesCardel/flp-2026-2-taller-02-cauca`, que trae la estructura de archivos con los módulos de `src/` y la fachada, la batería de pruebas, el `README.md` con la tabla de integrantes y las plantillas de los informes en `docs/`. El fork queda público. **Una entrega que no sea fork de ese repositorio pierde el 30 % de la nota.**
- **Se entrega pegando el enlace del fork en la actividad del taller en el campus virtual.** Nada más: no se aceptan archivos comprimidos (`.zip`, `.rar`, `.tar.gz`, etc.) ni archivos binarios (`.pdf`, `.docx`, imágenes insertadas). Solo se aceptan archivos de texto plano: código fuente (`.rkt`) y documentación en Markdown (`.md`).
- El código fuente se reparte en módulos dentro de la carpeta `src/`, con la fachada `interprete-cauca.rkt` y las pruebas `pruebas-cauca.rkt` en la raíz. Un intérprete escrito de corrido en un solo archivo no cumple con la organización pedida. Los nombres de los módulos y lo que va en cada uno están en *Estructura del repositorio*.
- **Todos los informes deben entregarse en formato Markdown dentro de la carpeta** `docs/` del repositorio. No se acepta otro formato ni archivos por fuera de esa carpeta. Guía: `https://docs.github.com/es/get-started/writing-on-github/getting-s`

tarted-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax.

- La notación matemática dentro de los archivos Markdown debe escribirse en **formato LaTeX**. Guía: `https://docs.github.com/es/get-started/writing-on-github/working-with-advanced-formatting/writing-mathematical-expressions`. No se aceptan imágenes insertadas para fórmulas.
- **Todos los diagramas** —árboles de sintaxis abstracta, cadenas de ambientes, trazas representadas gráficamente— deben generarse con **Mermaid** dentro de los Markdown. No se aceptan imágenes insertadas. Guía: `https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams`.
- El `README.md` del repositorio es lo que identifica al grupo: debe traer el nombre, el código y el correo institucional de cada integrante. Si el archivo falta, o si le falta alguno de esos datos, la entrega pierde el 20 % de la nota. Quien no aparezca ahí no cuenta como parte del grupo y su nota del taller es 0.0.
- No se permite copiar código de Internet ni de sus compañeros. Si se encuentra código copiado, el taller será anulado por completo para todas las partes involucradas. Los cambios en el repositorio deben ser significativos y no solo de formato o de comentarios; el historial de commits se revisa para ver qué aportó cada integrante.
- Las primeras líneas de cada archivo `.rkt` deben contener los nombres y códigos de los estudiantes:

```
;Autores: Nombre1 Codigo1, Nombre2 Codigo2
```

- Cada función implementada debe estar comentada indicando: nombre, contrato (entrada → salida) y propósito.

1.1. Indicadores de logro

1. **IL 1.1.7** Implementa los procedimientos de análisis léxico y sintáctico que derivan entre la representación concreta y la abstracta de una gramática BNF.
2. **IL 1.1.8** Utiliza herramientas para especificar analizadores léxicos y sintácticos a partir de una gramática BNF.
3. **IL 2.1.1** Construye ambientes para la ligadura de variables en un intérprete.
4. **IL 2.1.2** Implementa la evaluación de condicionales en un lenguaje interpretado.
5. **IL 2.1.3** Implementa clausuras y explica por qué un procedimiento debe recordar el ambiente de su definición.
6. **IL 2.1.4** Implementa la ligadura recursiva y justifica por qué `let` no basta para definir un procedimiento recursivo.

2. Contexto

El río que atraviesa el valle le da el nombre a **Cauca**, el lenguaje que va a construir en este taller. Cauca empieza donde termina la aritmética: números, booleanos, primitivas, un condicional y ligaduras locales. Con eso ya se puede calcular, pero no se puede abstraer: cada expresión que se quiera reutilizar hay que volverla a escribir.

Las dos piezas que convierten a Cauca en un lenguaje de verdad son los procedimientos y la recursión. La primera obliga a decidir qué recuerda un procedimiento cuando se lo invoca lejos de donde se definió, y esa decisión es la clausura. Con la recursión aparece un acertijo: para que un procedimiento pueda llamarse a sí mismo, su nombre tiene que estar en el ambiente donde se evalúa su propio cuerpo, y ese ambiente todavía no existe cuando se construye el procedimiento.

Aquí construye el intérprete completo de Cauca, lo extiende con primitivas nuevas y lo lleva hasta los procedimientos recursivos. Después explica por escrito, con notación formal, por qué lo que escribió funciona.

2.1. Estructura del repositorio

El intérprete va repartido en módulos dentro de `src/`; en la raíz quedan la fachada, las pruebas y el `README.md`, y los informes de las Partes 5 y 6 en `docs/`. El fork trae esta estructura armada: cada módulo con sus `require`, su `provide` y las funciones como huecos que levantan `eopl:error` con el mensaje `''Sin implementar''`, más el nombre y el contrato comentados encima. La entrega tiene exactamente estos archivos:

- `src/sintaxis.rkt`: especificación léxica, especificación gramatical, `sllgen:make-define-datatypes`, `scanner` y `parser`. No requiere ningún otro módulo.
- `src/valores.rkt`: el datatype `procval` con el constructor `cerradura`. Requiere `sintaxis.rkt`.
- `src/ambiente.rkt`: el datatype `ambiente` con sus tres variantes, `apply-env` y el ambiente inicial. Requiere `sintaxis.rkt` y `valores.rkt`.
- `src/primitivas.rkt`: `evaluar-primitiva` y las verificaciones de tipo y de cantidad de argumentos. Requiere `sintaxis.rkt`.
- `src/interprete.rkt`: `evaluar-programa`, `evaluar-expresion` y `apply-procedure`. Requiere los cuatro módulos anteriores.
- `src/interfaz.rkt`: el punto de entrada y el REPL. Requiere `sintaxis.rkt` e `interprete.rkt`.
- `interprete-cauca.rkt`: la fachada. Requiere los seis módulos y los reexporta con `(provide (all-from-out ...))`.
- `pruebas-cauca.rkt`: las pruebas con `rackunit`, que requieren solamente la fachada. Llega escrito en el fork y no se modifica.
- `docs/informe-ast.md` y `docs/informe-correccion.md`: los informes de las Partes 5 y 6.

- `README.md`: nombre, código y correo institucional de cada integrante.

Cada módulo exporta lo que define; con `(provide (all-defined-out))` basta. Un `define-datatype` declarado en un módulo se analiza con `cases` desde otro, y lo mismo vale para lo que genera SLLGEN: el parser, los constructores y los predicados cruzan de archivo sin trabajo adicional.

Las dependencias van en un solo sentido. Racket no carga dos módulos que se requieran entre sí: levanta el error `cycle in loading`. El nudo aparece con la recursión de la Parte 4. `apply-env` construye la clausura en el momento en que resuelve un nombre ligado por `letrec`, así que el ambiente necesita el constructor `cerradura`; y el campo del ambiente dentro de la clausura pediría declararse con `ambiente?`, con lo que el módulo de valores necesitaría al del ambiente. El corte se hace en un solo sentido: en `valores.rkt` ese campo se declara con un predicado que acepta cualquier valor, `(env (lambda (v) #t))`, y es `ambiente.rkt` el que requiere a `valores.rkt`. Se pierde la verificación de ese campo y de ningún otro. La salida sirve para cualquier campo cruzado y se usa únicamente cuando no queda otra forma de romper el ciclo.

Las partes del taller dicen qué implementar; el reparto en archivos es el de la lista anterior. Dos ubicaciones no se deducen del orden de las partes: `apply-procedure` vive en `interprete.rkt` porque llama a `evaluar-expresion`, y el ambiente inicial vive en `ambiente.rkt` porque `evaluar-programa` lo invoca.

3. Especificación de Cauca

3.1. Valores del lenguaje

- **Valores expresados:** números (enteros y flotantes), booleanos y procedimientos.
- **Valores denotados:** números (enteros y flotantes), booleanos y procedimientos.

Los procedimientos aparecen en las dos listas a partir de la Parte 3. Hasta la Parte 2 el lenguaje tiene solo números y booleanos.

3.2. Especificación léxica

- Los **comentarios** inician con `//` y llegan hasta el final de la línea.
- Los **espacios en blanco** se ignoran.
- Los **números** son enteros positivos, enteros negativos, flotantes positivos y flotantes negativos.
- Los **identificadores** son secuencias que inician con una letra, seguida de cero o más letras, dígitos, guiones bajos o signos de interrogación.

3.3. Gramática base

La gramática en forma BNF del lenguaje base es la siguiente. La información en los recuadros indica los nombres de los constructores del AST que deben usarse. Note que el condicional y las ligaduras locales cierran con la palabra reservada `end`: sin ella la gramática es ambigua cuando las expresiones se anidan.

```
<programa> ::= <expresion>
               a-program(exp)

<expresion>
           ::= <numero>
               lit-exp(dato)
           ::= <identificador>
               var-exp(id)
           ::= "true"
               true-exp()
           ::= "false"
               false-exp()
           ::= <primitiva> "(" {<expresion>}*(,) ")"
               prim-exp(prim, rands)
           ::= "if" <expresion> "then" <expresion>
               "else" <expresion> "end"
               if-exp(test, true-exp, false-exp)
           ::= "let" {<identificador> "=" <expresion>}*
               "in" <expresion> "end"
               let-exp(ids, rands, body)
```

```
<primitiva>
           ::= "+"      sum-prim()
           ::= "-"      minus-prim()
           ::= "*"      mult-prim()
           ::= "/"      div-prim()
           ::= "add1"   add-prim()
           ::= "sub1"   sub-prim()
           ::= ">"      mayor-prim()
           ::= ">="     mayorigual-prim()
           ::= "<"      menor-prim()
           ::= "<="     menorigual-prim()
           ::= "=="     igual-prim()
           ::= "not"    not-prim()
           ::= "and"    and-prim()
           ::= "or"     or-prim()
```

3.4. Ambiente inicial

El intérprete debe arrancar con un ambiente que contenga las siguientes ligaduras:

```
n = 10, m = 3, p = 7, q = 2, r = 5
```

3.5. Semántica de evaluación

- **lit-exp:** retorna el valor numérico.
- **var-exp:** busca el identificador en el ambiente actual y levanta error si no está ligado.
- **true-exp / false-exp:** retornan `#true` y `#false` respectivamente.
- **prim-exp:** evalúa los argumentos de izquierda a derecha y aplica la primitiva. Las primitivas aritméticas (`+`, `-`, `*`, `/`) son n-arias; `add1`, `sub1` y `not` son unarias; las relacionales, `and` y `or` son binarias.
- **if-exp:** evalúa la condición. Si el resultado es booleano, selecciona la rama correspondiente. Si **no** lo es, levanta un error con `eopl:error` indicando que la condición debe ser booleana.
- **let-exp:** evalúa las expresiones de los *rands* de izquierda a derecha en el ambiente actual, extiende el ambiente con las nuevas ligaduras y evalúa el cuerpo en el ambiente extendido.

Importante sobre if-exp: la evaluación del condicional debe ligar el resultado de evaluar la condición a una variable auxiliar (`test-value`), verificar con `boolean?` que sea un booleano y, en caso contrario, levantar `eopl:error`. No basta con pasar la condición evaluada directamente al `if` de Scheme, porque entonces cualquier número contaría como verdadero y el lenguaje perdería la distinción entre condición y dato.

4. Parte 1: Intérprete base (30 puntos)

Construya el intérprete completo de la gramática base, implementando:

1. La especificación léxica usando SLLGEN.
2. La especificación gramatical usando SLLGEN.
3. La generación automática de datatypes con `sllgen:make-define-datatypes`.
4. Las funciones `scanner` y `parser` usando las utilidades de SLLGEN.
5. El tipo de dato **ambiente** con `define-datatype`, con las variantes `ambiente-vacio` y `ambiente-extendido`.
6. La función `apply-env` para buscar variables en el ambiente.
7. La función `evaluar-programa` como punto de entrada.
8. La función `evaluar-expresion` que recorre el AST y evalúa cada tipo de expresión.
9. La función `evaluar-primitiva` que aplica las primitivas aritméticas, relacionales y lógicas.
10. El REPL usando `sllgen:make-rep-loop`.

Incluya al menos **5 ejemplos** del `scanner`, **5 ejemplos** del `parser` y **10 ejemplos** de ejecución que ejerciten todas las expresiones y primitivas de la gramática base, incluidos los casos de error.

```
// el area de un rectangulo, con el condicional que valida los lados
let base = 8
    altura = 5
in
  if and(>(base, 0), >(altura, 0))
  then *(base, altura)
  else 0
  end
end
// Resultado: 40
```

5. Parte 2: Extensión de primitivas (12 puntos)

Agregue las siguientes primitivas a la gramática:

`<primitiva>`		
	`::= "abs"`	`abs-prim()`
	`::= "mcd"`	`mcd-prim()`
	`::= "mcm"`	`mcm-prim()`
	`::= "promedio"`	`promedio-prim()`
	`::= "xor"`	`xor-prim()`
	`::= "implica"`	`implica-prim()`
	`::= "entre"`	`entre-prim()`

5.1. Semántica de las nuevas primitivas

- **abs:** unaria sobre números. Retorna el valor absoluto.
- **mcd:** n-aria sobre enteros, con $n \geq 2$. Retorna el máximo común divisor. Levanta error si algún argumento no es entero.
- **mcm:** n-aria sobre enteros, con $n \geq 2$. Retorna el mínimo común múltiplo.
- **promedio:** n-aria sobre números, con $n \geq 1$. Retorna la media aritmética.
- **xor:** binaria sobre booleanos. Retorna verdadero cuando exactamente uno de los dos argumentos lo es.
- **implica:** binaria sobre booleanos. Retorna falso únicamente cuando el primer argumento es verdadero y el segundo falso.
- **entre:** ternaria sobre números. `entre(x, a, b)` retorna verdadero cuando $a \leq x \leq b$.

Las primitivas booleanas deben verificar el tipo de sus argumentos y levantar `eopl:error` cuando reciban algo que no sea booleano; las numéricas, cuando reciban algo que no sea número. Incluya al menos **3 ejemplos** de ejecución por cada primitiva nueva, y al menos uno de error.

6. Parte 3: Procedimientos y clausuras (18 puntos)

Extienda la gramática con procedimientos y aplicación de procedimientos:

```
<expresion>
           ::= "proc" "(" {<identificador>}*(,) ")"
               <expresion> "end"
               proc-exp(ids, body)
           ::= "apply" <expresion>
               "(" {<expresion>}*(,) ")"
               app-exp(rator, rands)
```

La palabra reservada `apply` al inicio de la aplicación no es decorativa: sin ella, SLLGEN no puede decidir con un solo token de anticipación si un paréntesis abre una lista de argumentos o agrupa una expresión.

Por la misma razón, `<expresion>` no tiene una forma entre paréntesis y el operador de una aplicación no puede ser otra aplicación escrita en el mismo lugar: `apply (apply f (3)) (4)` no hace parte del lenguaje. Cuando un procedimiento retorna otro procedimiento, el intermedio se liga primero con `let` y después se aplica.

Otro detalle del léxico que conviene tener presente: el guion abre un número negativo, así que un identificador no puede llevarlo. `aplicar-n` se lee como el identificador `aplicar` seguido del número `-n`. Los nombres compuestos se escriben `aplicarn` o `aplicar_n`.

6.1. El valor procedimiento

Un procedimiento se representa con un datatype propio, que guarda los parámetros, el cuerpo sin evaluar y el ambiente vigente en el momento de la definición:

```
(define-datatype procval procval?
  (cerradura
    (ids (list-of symbol?))
    (body expresion?)
    (env ambiente?)))
```

6.2. Semántica

- **proc-exp:** no evalúa el cuerpo. Construye una `cerradura` con los parámetros, el cuerpo y el ambiente actual.
- **app-exp:** evalúa el operador, verifica con `procval?` que el resultado sea un procedimiento y levanta `eopl:error` si no lo es. Evalúa los argumentos de izquierda a derecha en el ambiente actual y luego invoca `apply-procedure`.
- **apply-procedure:** verifica que la cantidad de argumentos coincida con la de parámetros y levanta `eopl:error` cuando no coinciden. Evalúa el cuerpo en el ambiente de la clausura extendido con las ligaduras de los parámetros. **No** en el ambiente de la llamada.

6.3. Ejemplo de contraste

El siguiente programa distingue las dos decisiones posibles. Debe funcionar tal cual, y debe explicarlo en el informe de corrección:

```
let factor = 10
in
  let escalar = proc (v) *(v, factor) end
  in
    let factor = 1000
    in
      apply escalar (5)
    end
  end
end
```

El resultado correcto es 50. Si su intérprete responde 5000, está evaluando el cuerpo en el ambiente de la llamada en lugar del ambiente de la definición. Ese es justamente el error que la clausura evita.

Incluya al menos **6 ejemplos** de ejecución: procedimientos de cero, uno y varios parámetros, un procedimiento que recibe otro procedimiento como argumento, un procedimiento que retorna un procedimiento, y los dos casos de error.

7. Parte 4: Recursión y selección por valor (15 puntos)

7.1. 4.1 Expresión `letrec` (10 puntos)

```
<expresion>
           ::= "letrec" {<identificador>
               "(" {<identificador>}*(,) ")"
               "=" <expresion>}*
               "in" <expresion> "end"
             letrec-exp(proc-names, idss, bodies, body)
```

Semántica: las ligaduras de `letrec` son procedimientos que pueden invocarse a sí mismos y entre sí. El ambiente en el que se evalúa el cuerpo de cada procedimiento debe contener los nombres de **todos** los procedimientos declarados en el mismo `letrec`, incluido el suyo.

Para lograrlo agregue una tercera variante al datatype del ambiente, `ambiente-recursivo`, que guarde los nombres, las listas de parámetros y los cuerpos sin evaluar. La clausura se construye en el momento en que `apply-env` encuentra el nombre, y el ambiente que se le entrega es el propio `ambiente-recursivo`. Compare su solución con la Figura 3.9 de EOPL, sección 3.4.

Ejemplo:

```
letrec
  potencia (base, exp) = if ==(exp, 0)
                         then 1
                         else *(base, apply potencia (base, sub1(exp)))
                         end
```

```
in
  apply potencia (2, 10)
end
// Resultado: 1024
```

Incluya al menos **4 ejemplos**: un procedimiento recursivo simple, uno con dos parámetros, dos procedimientos mutuamente recursivos declarados en el mismo `letrec`, y uno que combine `letrec` con un procedimiento pasado como argumento.

7.2. 4.2 Expresión `case` (5 puntos)

```
<expresion>
           ::= "case" <expresion>
               {"of" <numero> "=>" <expresion>}+
               "otherwise" "=>" <expresion> "end"
             case-exp(exp, valores, acciones, default)
```

Semántica: evalúa la expresión de control una sola vez y compara su valor con los literales de las ramas, en orden. La primera coincidencia determina la acción que se evalúa; si ninguna coincide, se evalúa la rama `otherwise`. Si el valor de control no es un número, se levanta `eopl:error`.

La diferencia con `if` está en qué se compara: el condicional exige una expresión booleana en cada rama, mientras que `case` compara por igualdad contra literales fijos y evalúa la expresión de control una sola vez, por más ramas que tenga.

Ejemplo:

```
let dia = 3
in
  case dia
    of 1 => 100
    of 3 => 300
    of 7 => 700
    otherwise => 0
  end
end
// Resultado: 300
```

Incluya al menos **3 ejemplos**, uno de ellos con la rama `otherwise` activa.

8. Parte 5: Informe de AST y traza (10 puntos)

Entregue este informe en `docs/informe-ast.md`, que viene como plantilla en el repositorio base y se llena en el mismo archivo. La notación matemática va en LaTeX y **todos los diagramas en Mermaid**; no se aceptan imágenes insertadas ni documentos PDF.

8.1. 5.1 Árbol de sintaxis abstracta (5 puntos)

Dibuje el AST completo del siguiente programa de Cauca:

```
let umbral = promedio(n, m, p)
in
  letrec
    contar (k) = if <=(k, umbral)
                 then apply contar (add1(k))
                 else k
                 end
  in
    case apply contar (1)
      of 7 => abs(-(umbral, 7))
      otherwise => mcd(n, m)
    end
  end
end
```

Cada nodo debe llevar como etiqueta el nombre del constructor del AST (`let-exp`, `letrec-exp`, `case-exp`, `app-exp`, `prim-exp`, `lit-exp`, `var-exp`) y conectarse con sus hijos.

8.2. 5.2 Traza de evaluación (5 puntos)

Con la notación *value-of* de EOPL, haga la traza paso a paso del siguiente programa, partiendo del ambiente inicial ($n = 10$, $m = 3$, $p = 7$, $q = 2$, $r = 5$):

```
let desplazar = proc (x, d) +(x, d) end
in
  letrec
    acumular (i, total) = if >(i, m)
                          then total
                          else apply acumular
                                 (add1(i), apply desplazar (total, i))
                          end
  in
    apply acumular (1, q)
  end
end
```

Cada paso debe indicar la expresión que se evalúa, el ambiente en el que se evalúa —con la notación $[x = v_1, y = v_2, \ldots]$— y el resultado.

Incluya además un **diagrama de la cadena de ambientes en Mermaid** en el momento de la segunda llamada recursiva a `acumular`. Debe mostrar el `ambiente-recursivo`, el ambiente capturado por la clausura de `desplazar` y el ambiente vacío al final de la cadena, con las flechas dirigidas del ambiente que extiende hacia el ambiente extendido.

9. Parte 6: Informe de corrección (15 puntos)

Entregue este informe en `docs/informe-correccion.md`, que viene como plantilla en el repositorio base y se llena en el mismo archivo. Markdown, notación matemática en LaTeX y diagramas en Mermaid.

9.1. 6.1 Corrección de `apply-procedure` (5 puntos)

Demuestre que su implementación es correcta:

1. Establezca la especificación formal, con pre-condición y post-condición.
2. Demuestre que el cuerpo se evalúa en el ambiente de la clausura extendido con los parámetros, y no en el ambiente de la llamada. Use el ejemplo de contraste de la Parte 3 y muestre las dos cadenas de ambientes que corresponden a los resultados 50 y 5000.
3. Demuestre que la aplicación levanta error exactamente cuando el operador no es un procedimiento o cuando la cantidad de argumentos no coincide.

9.2. 6.2 Corrección de la ligadura recursiva (5 puntos)

Demuestre que su implementación de `letrec` es correcta:

1. Enuncie como invariante la propiedad que debe cumplir el ambiente en el que se evalúa el cuerpo de un procedimiento declarado con `letrec`.
2. Demuestre que `apply-env` sobre un `ambiente-recursivo` construye una clausura que satisface ese invariante.
3. Explique por qué la misma declaración escrita con `let` no lo satisface, y qué error concreto produce.
4. Argumente la terminación del ejemplo de `potencia` identificando la medida que decrece en cada llamada.

9.3. 6.3 Corrección de `case-exp` (5 puntos)

Demuestre que su implementación es correcta:

1. Establezca la especificación formal.
2. Demuestre por inducción estructural sobre la lista de ramas que se evalúa la acción de la primera coincidencia, que se evalúa `otherwise` cuando no hay ninguna, y que se levanta error cuando el valor de control no es un número.
3. Demuestre que la expresión de control se evalúa una sola vez, sin importar cuántas ramas tenga la expresión.
4. Justifique la terminación del recorrido sobre las ramas.

10. Pruebas

Escriba las pruebas con `rackunit` en `pruebas-cauca.rkt`, que requiere únicamente la fachada `interprete-cauca.rkt`. Las pruebas que trae el fork no se tocan: las suyas se agregan a continuación y requieren también la fachada, nunca un módulo de `src/` por separado. Cada módulo de `src/` exporta con `provide` lo que define y la fachada lo reexporta, de modo que las pruebas

ven todos los nombres sin requerir los módulos uno por uno. Ni los módulos ni la fachada contienen pruebas escritas. La documentación está en `https://docs.racket-lang.org/rackunit/`.

Las pruebas deben cubrir el `scanner`, el `parser` y la evaluación de todas las expresiones y primitivas, con al menos un caso de error por cada situación descrita en el enunciado: condición no booleana, variable no ligada, operador que no es procedimiento, cantidad de argumentos incorrecta, argumento de tipo equivocado en una primitiva y valor de control no numérico en `case`.

11. Rúbricas de evaluación

Las rúbricas que se van a utilizar para la evaluación consideran aspectos relacionados con las reglas del taller, la calidad de los informes y la realización de las implementaciones. En cada una se explica la asignación de puntos.

11.1. Sobre las reglas

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (3 pts)	**Nivel 2** (5 pts)
Nombres y códigos al inicio de cada archivo `.rkt` y en el `README.md`	No se cumple con esta regla	No están en todos los archivos o alguno de los estudiantes no incluye su código	Se encuentran en todos los archivos `.rkt` y en el `README.md` con nombres, códigos y correos
Funciones comentadas de acuerdo al enunciado	No se realizan comentarios	No se comentan todas las funciones o los comentarios no siguen el enunciado	Se comentan todas las funciones con nombre, contrato y propósito
Organización de la entrega en el repositorio	Entrega comprimidos, PDF, DOCX o imágenes insertadas, o los informes no están en `docs/`	El intérprete está en un solo archivo, los módulos no siguen los nombres del enunciado o están mezclados con binarios, o los informes no usan Markdown con Mermaid y LaTeX	Repositorio limpio con los seis módulos en `src/`, la fachada y las pruebas en la raíz, y los informes en Markdown con Mermaid y LaTeX dentro de `docs/`

Total: 15 puntos de reglas, que se descuentan del total obtenido.

11.2. Sobre el intérprete base (Parte 1)

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (5 pts)	**Nivel 2** (10 pts)
Especificación léxica y gramatical con SLLGEN	No se implementa o no compila	La especificación existe pero no cubre toda la gramática, o el `end` no está en `if` y `let`	Cubre toda la gramática base, genera los datatypes con `sllgen:make-define-datatypes` y hay 5 ejemplos de `scanner` y 5 de `parser`
Ambiente y `apply-env`	No se implementa o no usa `define-datatype`	Usa `define-datatype` pero `apply-env` no respeta el *shadowing* o no levanta error ante variable no ligada	Ambiente con las dos variantes, búsqueda correcta con *shadowing* y error explícito ante variable no ligada
Evaluación de expresiones y primitivas	No se implementa o la evaluación es incorrecta	Evalúa parte de las expresiones, o `if-exp` no verifica que la condición sea booleana	Evalúa todas las expresiones y primitivas de la base, `if-exp` verifica con `boolean?` y hay 10 ejemplos de ejecución

Total Parte 1: 30 puntos.

11.3. Sobre las primitivas nuevas (Parte 2)

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (3 pts)	**Nivel 2** (6 pts)
Implementación de las siete primitivas	No se implementan	Se implementan algunas, o la aridad no corresponde a la especificada	Las siete funcionan con la aridad especificada, incluidas las n-arias
Verificación de tipos y ejemplos	No hay verificación ni ejemplos	Verifica solo en algunas primitivas, o faltan ejemplos	Toda primitiva verifica el tipo de sus argumentos, levanta `eopl:error` y trae 3 ejemplos más uno de error

Total Parte 2: 12 puntos.

11.4. Sobre procedimientos y clausuras (Parte 3)

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (3 pts)	**Nivel 2** (6 pts)
Datatype `procval` y `proc-exp`	No se implementa	La clausura no guarda el ambiente de definición o el cuerpo se evalúa al construirla	`cerradura` guarda parámetros, cuerpo sin evaluar y ambiente de definición
`app-exp` y `apply-procedure`	No se implementa	Funciona en el caso simple pero no verifica el operador ni la cantidad de argumentos	Evalúa el cuerpo en el ambiente de la clausura extendido y levanta error en los dos casos previstos
Ejemplos de ejecución	No hay ejemplos	Menos de seis, o el ejemplo de contraste arroja 5000	Los seis ejemplos, incluidos procedimiento como argumento y como resultado, y el ejemplo de contraste arroja 50

Total Parte 3: 18 puntos.

11.5. Sobre recursión y `case` (Parte 4)

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (5 pts)	**Nivel 2** (10 pts)
Expresión `letrec`	No se implementa	Funciona con un solo procedimiento pero no con procedimientos mutuamente recursivos, o no usa `ambiente-recursivo`	Agrega la variante `ambiente-recursivo`, resuelve la recursión mutua y trae los cuatro ejemplos

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (2 pts)	**Nivel 2** (5 pts)
Expresión `case`	No se implementa	Selecciona la rama correcta pero evalúa la expresión de control más de una vez, o no valida que sea numérica	Evalúa el control una sola vez, selecciona la primera coincidencia, cae en `otherwise` y levanta error ante control no numérico

Total Parte 4: 15 puntos.

11.6. Sobre el informe de AST y traza (Parte 5)

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (2 pts)	**Nivel 2** (5 pts)
Árbol de sintaxis abstracta	No se entrega o no usa Mermaid	El árbol está incompleto o los nodos no llevan el nombre del constructor	El árbol completo en Mermaid, con los constructores como etiquetas y la anidación correcta
Traza y cadena de ambientes	No se entrega	La traza omite pasos o el diagrama no incluye el ambiente vacío ni el ambiente recursivo	Traza paso a paso con expresión, ambiente y resultado, y diagrama en Mermaid con el ambiente recursivo, la clausura y el ambiente vacío

Total Parte 5: 10 puntos.

11.7. Sobre el informe de corrección (Parte 6)

Criterio	**Nivel 0** (0 pts)	**Nivel 1** (2 pts)	**Nivel 2** (5 pts)
Corrección de `apply-procedure`	No se entrega	Describe la implementación sin especificación formal, o no contrasta las dos cadenas de ambientes	Especificación formal, demostración del ambiente de la clausura con las dos cadenas y los dos casos de error
Corrección de la ligadura recursiva	No se entrega	Enuncia el invariante sin demostrarlo, o no explica por qué `let` falla	Invariante enunciado y demostrado, explicación del fallo con `let` y argumento de terminación
Corrección de `case-exp`	No se entrega	Demostración incompleta o sin argumento de terminación	Especificación, inducción sobre las ramas, evaluación única del control y terminación

Total Parte 6: 15 puntos.

12. Resumen de puntuación

Componente	Puntos
Parte 1: Intérprete base	30
Parte 2: Extensión de primitivas	12
Parte 3: Procedimientos y clausuras	18
Parte 4: Recursión y selección por valor	15
Parte 5: Informe de AST y traza	10
Parte 6: Informe de corrección	15
Total	**100**

Nota: los puntos de reglas, hasta 15, se descuentan del total obtenido. Si el `README.md` no lleva nombres, códigos y correos, se descuenta además el 20 % de la nota, y si el repositorio entregado no es un fork del repositorio base, el 30 %.

Su entrega será valorada a partir de las rúbricas consignadas previamente; cada una contiene una lista de criterios y niveles de desempeño. Para obtener la máxima calificación posible debe cumplir las especificaciones del nivel más alto en cada criterio.

$$\text{Nota taller} = 5{,}0 \times \frac{\text{puntos obtenidos} - \text{descuento reglas}}{100}$$

El docente puede requerir la sustentación del taller. En ese caso se aplica un factor de sustentación entre 0 y 1, individual, que multiplica la nota.

13. Referencias

- Friedman, D. P., & Wand, M. *Essentials of Programming Languages*, 3.ª ed., MIT Press, 2008. Sección 3.2 (el lenguaje LET), sección 3.3 (procedimientos y clausuras, Figura 3.7) y sección 3.4 (procedimientos recursivos, Figura 3.9).
- The Racket Guide, *Parsing with SLLGEN*, en la documentación del lenguaje `eopl`.
- RackUnit, *Unit Testing*: `https://docs.racket-lang.org/rackunit/`.